Machine Learning-based Late Delivery Risk Prediction in Global Supply Chain Operations

Author: Shridhar Suhas Jadhav | **Company:** Unified Mentor (internship project) | **Case organisation:** APL Logistics (KWE Group) | **Date:** November 2026

Live dashboard: https://late-delivery-analyzer.streamlit.app/

Abstract

Late deliveries cause SLA breaches, penalties and customer churn, and handling them after they happen is costly. This paper presents a predictive system that estimates, before an order ships, the probability that it will be delivered late. Using 180,519 order records with 40 attributes from APL Logistics, we remove cancelled and fraud-flagged orders, eliminate columns that leak the outcome, engineer five pre-shipment features, and train three classifiers: Logistic Regression, Random Forest and XGBoost. On a held-out test set of 34,553 orders the models reach a ROC-AUC of 0.745, 0.748 and 0.746, with precision near 0.88 and recall near 0.54 at a 0.5 threshold. Each order receives a late-delivery probability, a Low, Medium or High Risk category, and a SHAP-based explanation of its key risk drivers. The High Risk band contains 57,801 orders (33.5%) of which 90.0% were actually late. A central finding is that shipping mode explains almost all of the predictable signal: a one-line rule based only on shipping mode reaches an AUC of 0.742, so the machine-learning models add only about 0.004 AUC. First Class orders were late in every one of 26,513 cases, and took 2.00 days against a 1-day schedule. The system is delivered as a four-module Streamlit dashboard with shipping-mode, region, segment and risk-threshold filters, giving operations teams a ranked action queue. We discuss limitations of the data and recommend operational changes and additional data sources.

Keywords: late delivery prediction, supply chain analytics, XGBoost, SHAP explainability, risk scoring, Streamlit

1. Introduction

1.1 Background and motivation

In global logistics networks a late delivery is expensive for three reasons. It can breach a service-level agreement (SLA) and trigger penalties, it damages customer trust and increases churn, and it is usually discovered only after the damage is done. Reactive

handling, such as expediting a parcel that is already late or apologising to a customer, costs far more than acting early. If a team knows in advance which orders are likely to be late, it can reroute, reprioritise, or warn the customer while there is still time.

A descriptive analysis of past delays explains why delays happened. Operations teams, however, also need a forward-looking answer to a different question: *which orders are likely to be delayed before they are shipped?* This project provides that capability.

1.2 Problem statement

APL Logistics faces unpredictable shipment delays, a high operational cost of last-minute fixes, and limited ability to prioritise high-risk orders. The organisation lacks a predictive system that flags late-delivery risk early, quantitative risk scores for operational planning, and explainable insight into delay drivers.

1.3 Objectives

1. Build a supervised model that predicts late delivery using only information available before shipping.
2. Produce, for every order, a late-delivery probability between 0 and 1 and a Low, Medium or High Risk category.
3. Identify and explain the main drivers of delay, both globally and for an individual order.
4. Deliver the results as an interactive dashboard that supports risk-based prioritisation.

1.4 Contributions

- A leakage-aware modelling pipeline with a clearly documented list of excluded columns.
- A transparent comparison of three models, including a shipping-mode-only baseline that puts the added value of machine learning into perspective.
- Four decision-oriented KPIs: Late Delivery Probability, Risk Category, Key Risk Drivers and a High-Risk Order List.
- A deployed four-module Streamlit dashboard that serves both aggregate monitoring and single-order scoring.

2. Data

2.1 Dataset description

The dataset contains **180,519 order lines and 40 columns**. The columns describe payment type, scheduled and actual shipping days, profit and sales values, product and category information, customer identity and location, market and order region, discount and

quantity, order status, and shipping mode. The target variable is `Late_delivery_risk` (1 = late, 0 = not late). In the raw data 98,977 orders (54.8%) are late and 81,542 (45.2%) are not.

2.2 Data quality

Data quality is high. Only 11 cells are empty in the whole table (8 in `Customer Lname` and 3 in `Customer Zipcode`), both in columns that are not used for modelling, and there are no duplicate rows.

2.3 Leakage and exclusions

A prediction made *before shipping* must not use information that only exists *after* shipping. Three columns were removed for this reason:

Column removed	Reason
Delivery Status	Contains the outcome itself. All 98,977 "Late delivery" rows have `Late_delivery_risk` = 1, and every other status (Advance shipping 41,592; Shipping on time 32,196; Shipping canceled 7,754) has 0.
Days for shipping (real)	The actual duration is known only after delivery.
Order Status	Statuses such as COMPLETE or CLOSED are assigned after the order is processed.

We also removed the 7,754 orders with status CANCELED or SUSPECTED_FRAUD, because they never ship and therefore cannot be on time or late. This leaves **172,765 orders, of which 57.3% are late**.

A further 23 columns were dropped because they identify individuals (names, street, zip code, customer and order-customer identifiers), have very many distinct values (cities, states, countries, product name, latitude and longitude), or duplicate other columns (benefit per order, sales per customer, order item total, order profit per order, product price, category and department identifiers). Market and order region were retained as lower-cardinality geographic descriptors. These exclusions are a design judgement rather than the result of an ablation test.

3. Exploratory Data Analysis

3.1 Target balance

After cleaning, 57.3% of orders are late and 42.7% are not. The imbalance is mild, so class weighting is sufficient and oversampling such as SMOTE was not required.

3.2 Late rate by shipping mode

Shipping mode is by far the strongest pattern in the data, and it runs against intuition: faster services are late more often.

Shipping mode	Orders	Late rate	Scheduled days	Mean actual days
First Class	26,513	100.0%	1	2.00
Second Class	33,806	79.8%	2	3.99
Same Day	9,293	47.9%	0	0.48
Standard Class	103,153	39.8%	4	3.99

Every First Class order was late, with an average actual duration of exactly 2.00 days against a promised 1 day. Second Class orders take about 4 days against a 2-day promise. Standard Class is closest to its promise. The scheduled number of days is fixed per mode (Same Day 0, First Class 1, Second Class 2, Standard Class 4), so scheduled days and shipping mode carry almost the same information.

3.3 Other dimensions

Dimension	Late-rate range (cleaned data)
Market (5 markets)	56.8% (Africa) to 57.7% (Europe)
Customer segment (3 segments)	57.1% (Corporate) to 57.6% (Home Office)
Order region (23 regions)	51.6% (Canada) to 60.1% (Central Africa)

Market and customer segment barely differ. Order region varies more, with a spread of about 8.5 percentage points, which motivated a regional congestion feature.

4. Methodology

4.1 Pipeline overview

The workflow is: cleaning, leakage removal, feature engineering, stratified train/test split, preprocessing, model training with imbalance handling, evaluation, risk scoring, explainability, and deployment.

4.2 Feature engineering

Five features were created from information known before shipping.

Feature	Definition	Intent
sched_days	Scheduled shipping days	Planned lead time
is_express	1 if mode is First Class or Same Day, else 0	Mode risk flag
shipping_pressure	Quantity / (scheduled days + 1)	Shipping pressure index: many items in little time
order_complexity	Quantity × product price × (1 + discount rate) / 100	Order complexity score: large, costly, discounted orders
region_congestion	Late rate of the order region, computed on the training set only	Regional congestion indicator

The regional congestion value is a target encoding. To avoid leakage it was calculated from training data only, and unseen regions fall back to the overall training late rate. The final input set has 17 features: 6 categorical (shipping mode, market, order region, customer segment, payment type, department name) and 11 numeric.

4.3 Train/test split and preprocessing

The data were split 80/20 with stratification on the target and a fixed random seed (42): 138,212 training orders and 34,553 test orders. Numeric features were standardised, and categorical features were one-hot encoded, giving 61 model input columns. The preprocessing is fitted on training data only and applied to test data.

4.4 Handling class imbalance

Class weights were used instead of resampling: `class_weight='balanced'` for Logistic Regression and Random Forest, and `scale_pos_weight` equal to the ratio of negative to positive training examples for XGBoost.

4.5 Models

- **Logistic Regression** (baseline, interpretable): up to 1,000 iterations.
- **Random Forest** (Breiman, 2001): 200 trees, maximum depth 12, minimum 20 samples per leaf.
- **XGBoost** (Chen and Guestrin, 2016): 300 trees, maximum depth 5, learning rate 0.08, subsample 0.8, column subsample 0.8.

Hyperparameters were fixed from reasonable defaults and were not tuned by search. A single stratified split was used and no cross-validation was performed.

4.6 Evaluation metrics

Metric	Business meaning
ROC-AUC	Overall ability to rank late orders above on-time orders (Fawcett, 2006).
Precision	Of the orders flagged as late, the share that really were late. High precision means few false alarms.
Recall	Of all truly late orders, the share that were caught.
F1 score	Harmonic mean of precision and recall.
Confusion matrix	Counts of correct predictions, false alarms and missed delays.

Precision, recall and F1 are reported at a 0.5 probability threshold unless stated otherwise.

4.7 Risk probability and categories

The final model outputs a late-delivery probability for each order. Orders are placed into **Low Risk** (probability up to 0.40), **Medium Risk** (above 0.40 up to 0.70) and **High Risk** (above 0.70). These cut-offs were chosen for operational readability, not optimised against a cost function.

4.8 Explainability

We used TreeSHAP (Lundberg and Lee, 2017) on the XGBoost model. Global importance is the mean absolute SHAP value over 2,000 randomly drawn test orders, with one-hot columns summed back to their original feature. For a single order, SHAP values give the contribution of each feature to the log-odds of being late, which is what the dashboard shows as the key contributing factors.

5. Results

5.1 Model comparison

Model	ROC-AUC	Precision	Recall	F1
Logistic Regression	0.745	0.886	0.540	0.671
Random Forest	0.748	0.886	0.540	0.671
XGBoost	0.746	0.882	0.543	0.672
Shipping-mode-only rule	*0.742*	*0.886*	*0.540*	*0.671*

All three models are very close, within 0.003 AUC of each other. The last row is a simple baseline: each order is scored with the late rate of its shipping mode measured on the training set, and flagged as late if that rate is at least 0.5 (that is, First Class and Second Class orders). On the same test set this rule matches Logistic Regression and Random Forest at the 0.5 threshold. With a single train/test split, the differences between the models are too small to claim that any one is better.

XGBoost was chosen as the final model for practical reasons: its saved file is compact (about 1 MB), it is fast, and it has native TreeSHAP support. It was not chosen because of a statistically established accuracy advantage.

5.2 Confusion matrix (XGBoost, 0.5 threshold)

	Predicted on time	Predicted late
Actually on time	13,316	1,442 (false alarms)
Actually late	9,038 (missed)	10,757

The model is conservative: when it says an order will be late it is usually right (precision 0.882), but it misses 46% of late orders (recall 0.543). The missed delays are overwhelmingly Standard Class orders, whose lateness cannot be predicted from the available pre-shipment attributes.

5.3 Choosing the alert threshold

Threshold	Precision	Recall
0.3	0.592	0.938

Threshold	Precision	Recall
0.4	0.835	0.581
0.5	0.882	0.543
0.6	0.886	0.540
0.7	0.891	0.521

The threshold is a business choice. A low threshold (0.3) catches 93.8% of late orders but about 41% of the alerts are false alarms, which suits cheap actions such as an automatic customer notification. A high threshold (0.7) gives more trustworthy alerts but catches only 52.1% of late orders, which suits costly interventions such as expediting. The dashboard exposes this trade-off through a risk-threshold slider.

5.4 Risk categories

The final model was applied to all 172,765 orders. The average predicted probability is 0.519 (median 0.358).

Risk category	Orders	Share	Actually late	Sales value
Low Risk	103,673	60.0%	38.6%	21.10 million
Medium Risk	11,291	6.5%	61.3%	2.37 million
High Risk	57,801	33.5%	90.0%	11.74 million

The categories separate late and on-time orders well: nine in ten High Risk orders were really late. High Risk orders carry 11.74 million in sales value, 33.3% of the total of 35.21 million, and weighting each by its probability gives an expected 10.19 million of sales exposed to delay (values in the dataset's currency units). The bands closely follow shipping mode: the High Risk band consists of 31,282 Second Class and 26,513 First Class orders (plus 6 others), the Low Risk band of 99,613 Standard Class and 4,059 Same Day orders, and the Medium Risk band of Same Day (5,229), Standard Class (3,539) and Second Class (2,523) orders.

One caution applies. Probabilities were assigned to all orders, including the 80% used for training, so the band statistics above are in-sample. The accuracy figures in Sections 5.1 to 5.3 are the held-out test results and are the ones to quote as model performance.

5.5 Key risk drivers

Rank	Feature	Mean |SHAP|
1	Shipping mode	1.563
2	Scheduled days	0.636
3	Order region	0.049
4	Order complexity	0.037
5	Sales	0.034
6	Payment type	0.034
7	Order item discount	0.033
8	Order item profit ratio	0.029

Shipping mode and scheduled days dominate, and every other feature contributes less than 0.05. This agrees with the exploratory analysis and with the baseline result in Section 5.1.

The other features are not entirely useless. Within a single shipping mode the model still ranks late above on-time orders, with an AUC of 0.678 for Same Day, 0.637 for Second Class and 0.604 for Standard Class. These values were computed on all orders and are therefore optimistic, and they show a weak signal rather than a strong one.

5.6 Explaining an individual order

For the highest-risk order in the explanation sample (First Class, Africa market, North Africa region, predicted probability close to 1.0), the largest contributions to the log-odds of lateness were shipping mode (+7.56), scheduled days (+1.55), order region (+0.16), product price (+0.13) and shipping pressure (+0.11). An operations analyst can read this directly: the order is high risk almost entirely because it was booked as First Class with a 1-day schedule. Positive values push toward late and negative values push toward on time.

6. System Implementation: Streamlit Dashboard

6.1 Architecture

The scored orders and the model are packaged for a Streamlit application deployed on Streamlit Community Cloud at https://late-delivery-analyzer.streamlit.app/. The deployment contains seven files:

File	Purpose
app.py	Dashboard code
scored_orders.csv.gz	Scored orders for the aggregate pages (about 2.1 MB, compressed)
xgb_model.json	Trained XGBoost model stored as plain JSON (about 1 MB)
preprocess.json	Scaling constants, category lists and regional congestion values
metrics.json	Model evaluation results
risk_drivers.csv	Global key risk drivers
requirements.txt	Libraries installed at deployment

The model is stored as JSON and the preprocessing recipe as a separate file, so the application does not depend on the scikit-learn version used for training, which avoids a common source of deployment failures with pickled models. In testing, the application's own preprocessing and prediction code reproduced the training pipeline's probabilities exactly on 300 sampled orders.

6.2 Dashboard modules

Module	Content
Delay Risk Overview	Overall risk distribution, high-risk order count, average late probability, model ROC-AUC and the Key Risk Drivers chart.
Order-Level Risk Prediction	A form for a single order. Returns the Late Delivery Probability, the Risk Category and the key contributing factors.
Region & Mode Risk Analysis	A heatmap of average risk by region and shipping mode, and predicted versus actual late rate by mode.
Operations Action Panel	A prioritised list of orders above the risk threshold, ranked by probability or by sales at risk, with a CSV download of the High-Risk Order List.

6.3 User controls

The sidebar provides filters for shipping mode, market, order region and customer segment, plus a risk-threshold slider that controls which orders count as high risk.

6.4 Key performance indicators

KPI	Description	Value in this study
Late Delivery Probability	Likelihood of delay, 0 to 1	Mean 0.519
Risk Category	Priority for intervention	57,801 High, 11,291 Medium, 103,673 Low
Key Risk Drivers	Shipping, region or order factors	Shipping mode, then scheduled days
High-Risk Order List	Operational action queue	57,801 orders

7. Discussion and Recommendations

7.1 What the results mean

The analysis points to a structural problem rather than a prediction problem. Late deliveries are driven mostly by the gap between what each shipping mode promises and what it delivers. Machine learning confirms and quantifies this and ranks individual orders slightly better than a mode rule, but the dominant lever is operational.

7.2 Recommendations

1. **Reset or fix the First Class promise.** All 26,513 First Class orders were late, averaging 2.00 days against a 1-day schedule. Either promise 2 days, or investigate why the 1-day process fails. Until then, every First Class order should be treated as high risk.
2. **Review the Second Class promise.** About 80% are late, with actual duration near 4 days against a 2-day schedule. Align the promise with real capability or add capacity.
3. **Treat Standard Class as a volume problem.** Its 39.8% late rate applies to 103,153 orders, roughly 41,000 late deliveries, and these are hard to predict from order attributes. Improvement here needs operational data (see 8.2), not a better algorithm.
4. **Use the threshold slider by action cost.** Use a low threshold (about 0.3) for low-cost proactive notifications, and a high threshold (0.7) for expensive interventions such as expediting.

5. **Prioritise by value as well as probability.** The Action Panel can rank by sales at risk so that large, likely-late orders are handled first.
6. **Communicate proactively.** Send customers early notices for High Risk orders to reduce complaints and churn.
7. **Monitor.** Log predicted risk against the actual outcome and retrain periodically.

8. Limitations and Future Work

8.1 Limitations

- **Limited predictive ceiling.** ROC-AUC is about 0.75, and shipping mode alone gives 0.742. The models add little beyond mode.
- **Very regular data.** Mean durations such as exactly 2.00 days for First Class suggest highly standardised, possibly simulated data. Results should be validated on real operational records before being relied on.
- **No time information.** The dataset has no order dates or timestamps, so temporal validation, seasonality and drift cannot be assessed.
- **Single split and fixed hyperparameters.** No cross-validation or hyperparameter search was done, and confidence intervals were not computed.
- **Calibration and thresholds.** Probability calibration was not evaluated, and the 0.4 and 0.7 category cut-offs were chosen by judgement rather than by cost minimisation.
- **In-sample risk bands.** Band statistics in Section 5.4 include training orders.
- **Excluded variables.** Excluding columns such as city, state and product was a judgement and was not tested individually.

8.2 Future work

- Add operational data: order and dispatch timestamps, carrier, warehouse, route, weather, port congestion and holiday calendars.
- Compare models with cross-validation and statistical tests, and tune hyperparameters.
- Calibrate the probabilities and choose thresholds from a cost model.
- Track predictions in production and add drift monitoring.
- Extend the output from a yes/no risk to a predicted delay duration.

9. Conclusion

This project moves APL Logistics from reactive delay handling toward predictive risk intelligence. A leakage-aware pipeline produced a late-delivery probability, a risk category and an explanation for every order, delivered through a four-module dashboard. The held-out ROC-AUC is about 0.75, and 90% of orders in the High Risk band were really late. The most useful finding is also the simplest: shipping mode, together with its fixed scheduled days, explains nearly all of the predictable delay, and a First Class promise that was never met in this data is the clearest operational target. Future gains will come mainly from richer operational data rather than from more complex models.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Unified Mentor. *Machine Learning–based Late Delivery Risk Prediction in Global Supply Chain Operations*, project brief, APL Logistics (KWE Group).

Appendix A: Reproducibility

The analysis was carried out in a Python notebook (pandas, scikit-learn, XGBoost, SHAP, matplotlib) in Google Colab. All random operations use a fixed seed of 42. Library versions used for training are written to `requirements.txt` by the notebook, and the deployed model files (`xgb_model.json` , `preprocess.json`) are produced by the same notebook.